ECG Beat Classification using ML

Nadav Robinson
ID: 212544779

Yuval Cohen
ID: 323071043

Michael Babiy
ID: 323073734

July 20, 2025

Abstract

This report details the development and evaluation of machine learning models for the automated classification of electrocardiogram (ECG) beats. The objective is to accurately classify heartbeats into five standard arrhythmia categories defined by the Association for the Advancement of Medical Instrumentation (AAMI): Normal (N), Supraventricular (S), Ventricular (V), Fusion (F), and Unknown/Paced (Q). Using the MIT-BIH Arrhythmia Database, we extracted a 16-dimensional feature vector for each heartbeat, comprising statistical, wavelet, and morphological features. We implemented and compared three ensemble-based classifiers from scratch: a Decision Tree, a Random Forest, and an AdaBoost model, validating them against their scikit-learn counterparts. Our results demonstrate the viability of using feature-based machine learning for reliable ECG beat classification, with the Random Forest and AdaBoost models showing superior performance over a single Decision Tree.

1 Introduction

Cardiac arrhythmia detection from ECG signals is a critical task in automated healthcare diagnostics. Manual annotation by expert cardiologists is time-consuming, expensive, and prone to human error, which motivates the use of machine learning to automate and scale beat classification. We use the MIT-BIH Arrhythmia Database, a standard benchmark in the field, to classify beats into five categories defined by the AAMI standard: Normal (N), Supraventricular (S), Ventricular (V), Fusion (F), and Unknown/Paced (Q).

The input to our system is a one-dimensional ECG signal from Lead I (sampled at 360 Hz). Using the expert-annotated R-peaks and beat types, we extract fixed-length heartbeat windows centered around each annotated R-peak. The output of our system is a predicted beat class label (N, S, V, F, or Q) using a trained machine learning classifier. Our primary motivation is to build a lightweight, interpretable model that can serve as a baseline or even a production candidate for low-resource clinical settings, mobile monitoring, or embedded health devices, where computational efficiency is paramount.

2 Related Work

The automatic classification of ECG signals has been a significant area of research. The MIT-BIH Arrhythmia Database is the most widely used dataset for developing and bench-

marking these algorithms. Early approaches focused heavily on feature engineering, extracting various morphological and timing-based features from the raw ECG signal to train classical machine learning models like Support Vector Machines (SVMs) and Decision Trees [5].

More recently, deep learning models, especially Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), have achieved state-of-the-art performance by learning hierarchical features directly from the raw or transformed ECG data [3, 4, 6]. While these models are powerful, they often function as "black boxes," making them difficult to interpret—a significant drawback in clinical applications where understanding the model's reasoning is crucial. Our work aims to find a middle ground by using carefully engineered, intuitive features combined with interpretable ensemble models. This approach, while perhaps not matching the peak accuracy of the most complex deep learning architectures, provides a transparent and efficient alternative [2].

3 Dataset and Features

3.1 Dataset Source

We utilize the MIT-BIH Arrhythmia Database [1], a publicly accessible and well-annotated ECG dataset. This database comprises 48 half-hour, two-channel ECG recordings from 47 different subjects. Each record contains ECG waveforms sampled at a rate of 360 Hz, along with beat-by-beat annotations from expert cardiologists detailing R-peak locations and beat labels.

3.2 Beat Extraction and Segmentation

From all 48 records, we extracted individual heartbeats using the 'ann.sample' values, which mark the sample index of each annotated R-peak. For each beat, we isolated a 0.5-second window (180 samples) centered on the R-peak (0.25 seconds before and 0.25 seconds after). This fixed-length segment captures the complete P-QRS-T waveform, which is essential for classification.

3.3 Feature Extraction

For each heartbeat segment, we extracted a 16-dimensional feature vector. These features are designed to capture the segment's statistical properties, timing relative to other beats, frequency components, and overall shape.

- **Basic Statistical Features (4 features):** These provide a general overview of the heartbeat's amplitude and complexity.
  - *Segment Length:* The number of data points in the segment.
  - *Mean:* The average amplitude, indicating baseline shifts.
  - *Standard Deviation:* Measures the signal's "wiggle," or variability.
  - *Range:* The difference between the maximum and minimum amplitude.
- **RR Interval Features (2 features):** These capture the rhythm of the heart by measuring the time between beats.

  - *Current RR Interval:* The time between the current R-peak and the next one.
  - *Previous RR Interval:* The time between the previous R-peak and the current one.

- **Wavelet Decomposition (8 features):** A 3-level discrete wavelet transform (DWT) with a Daubechies-1 wavelet was used to decompose the signal into different frequency bands. This is like separating a song into its bass, mid-range, and treble tracks. The approximation coefficients represent the slow, main trend of the heartbeat, while the detail coefficients capture faster, more subtle changes. For each of the 4 resulting coefficient arrays (1 approximation, 3 detail), we calculated the mean and standard deviation.

- **Shape Descriptors (2 features):** These describe the shape of the heartbeat's amplitude distribution.
  - *Skewness:* Measures the asymmetry or "tilt" of the waveform. A perfectly symmetric wave has a skewness of zero.
  - *Kurtosis:* Measures the "peakiness" of the waveform. A high kurtosis indicates a sharp, pointy peak, while a low kurtosis suggests a more rounded or flat top.

3.4 Beat Labeling

We used the original beat symbols from the 'ann.symbol' field and mapped them to the 5-class AAMI standard using a predefined dictionary to create our target labels for supervised learning.

4 Methods

We framed the classification of ECG beats as a supervised learning problem: given a feature vector $\mathbf{x} \in \mathbb{R}^{16}$, predict the AAMI-class label $y \in \{\mathrm{N}, \mathrm{S}, \mathrm{V}, \mathrm{F}, \mathrm{Q}\}$. We implemented and evaluated three ensemble-based algorithms from scratch: **Decision Tree**, **Random Forest**, and **AdaBoost**.

4.1 Decision Tree

A *Decision Tree* is a non-parametric algorithm that learns a hierarchy of if-then rules. At each node, it selects a feature and a threshold that best splits the data to increase class purity, measured by Gini impurity:

$$G(t) = 1 - \sum_{k=1}^{K} p_k^2$$

where p_k is the proportion of class k samples at node t. Our from-scratch implementation supports multi-class classification, a maximum tree depth, and a minimum number of samples per split to prevent overfitting. We validated its correctness against scikit-learn's 'DecisionTreeClassifier'.

4.2 Random Forest

A *Random Forest* is an ensemble method that reduces the high variance of individual decision trees by training many of them on different bootstrap samples of the data (bagging). At each split, only a random subset of features is considered, which de-correlates the trees. The final prediction is a majority vote over all trees:

$$\hat{y} = \text{majority\_vote}(f_1(\mathbf{x}), f_2(\mathbf{x}), \ldots, f_B(\mathbf{x}))$$

4.3 AdaBoost (Adaptive Boosting)

AdaBoost is a boosting algorithm that builds an ensemble of weak learners (typically shallow decision trees), trained sequentially. Each new learner focuses on the mistakes made by the previous ones by updating the weights of the training examples. We implemented AdaBoost from scratch using the SAMME (Stagewise Additive Modeling using a Multi-class Exponential loss function) algorithm, which is designed for multi-class classification.

At each iteration t, for a dataset with K classes, AdaBoost performs the following steps:

1. Trains a weak classifier $h_t(\mathbf{x})$ on the data using the current sample weights w_i.
2. Computes the weighted error ε_t:

$$\varepsilon_t = \frac{\sum_{i=1}^{n} w_i \cdot \mathbb{I}(h_t(x_i) \neq y_i)}{\sum_{i=1}^{n} w_i}$$

   where $\mathbb{I}(\cdot)$ is the indicator function.
3. Calculates the classifier weight, α_t, which measures its contribution to the final ensemble. This is the key step in the SAMME algorithm:

$$\alpha_t = \ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right) + \ln(K-1)$$

4. Updates the sample weights, increasing the weights of misclassified samples:

$$w_i \leftarrow w_i \cdot \exp(\alpha_t \cdot \mathbb{I}(h_t(x_i) \neq y_i))$$

   The weights are then normalized to sum to 1.

The final prediction for a new sample $\mathbf{x}$ is determined by a weighted majority vote from all T weak learners. For each class $k \in \{1, \ldots, K\}$, the model calculates a score by summing the weights of the classifiers that predicted that class. The final class is the one with the highest score:

$$\hat{y} = \underset{k}{\text{argmax}} \left(\sum_{t=1}^{T} \alpha_t \cdot \mathbb{I}(h_t(\mathbf{x}) = k) \right)$$

This approach allows AdaBoost to effectively handle multi-class problems and improve performance on underrepresented classes by focusing on difficult-to-classify examples.

5 Experiments and Results

5.1 Experimental Setup

We subsampled 5000 beats from the full extracted dataset for rapid iteration, stratified into 4000 training and 1000 test samples (class distribution: N 80.05%, Q 9.72%, V 7.05%, S 2.45%, F 0.73%). An initial baseline used default or simple settings (e.g., untuned tree depth, full bootstrap sampling, full feature set). We then performed a constrained hyperparameter search focusing on parameters with the largest expected bias–variance impact for tree ensembles:

- **Decision Tree:** max_depth $\in \{2, 3, 4, 5, 6\}$; min_samples_split $\in \{2, 5, 10\}$.
- **Random Forest:** tree depth (3–6), min_samples_split $\{2, 5, 10\}$, bootstrap sample fraction $\{50\%, 60\%, 80\%, 100\%\}$, feature subsampling $\{\sqrt{d}, 50\%, 60\%, 80\%\}$.
- **AdaBoost (SAMME):** weak learner depth $\{1, 2, 3, 4\}$, min_samples_split $\{2, 5\}$, up to 100 iterations.

Selections were guided by validation accuracy and minority-class (S, F) F1 trend (even when absolute F1 remained low due to scarcity). Final tuned settings:

Tuned Hyperparameters

Decision Tree: max_depth = 4, min_samples_split = 5
Random Forest: max_depth = 4, min_samples_split = 5, 60% bootstrap, 60% features
AdaBoost: depth-3 weak trees, min_samples_split = 2

All reported test metrics below reflect these tuned configurations.

5.2 Quantitative Results

The performance of our models was evaluated using accuracy, precision, recall, and F1-score. The results on the test set are presented below. For each model, the performance metrics and confusion matrix are displayed side-by-side for a comprehensive view.

Figure 1: Decision Tree Results and Confusion Matrix

Class	Prec.	Recall	F1	Supp.
F	0.00	0.00	0.00	7
N	0.91	0.99	0.95	801
Q	0.88	0.62	0.73	97
S	0.17	0.04	0.07	24
V	0.80	0.68	0.73	71
Acc.		0.90 (1000)		

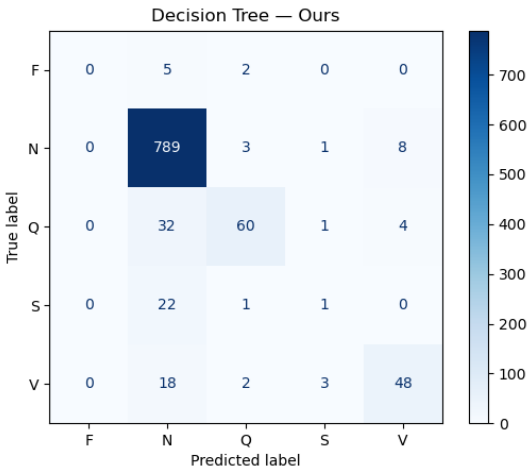


Figure 2: Decision Tree Depth vs Accuracy. This plot shows how varying the tree depth affects model accuracy, demonstrating where overfitting may occur.

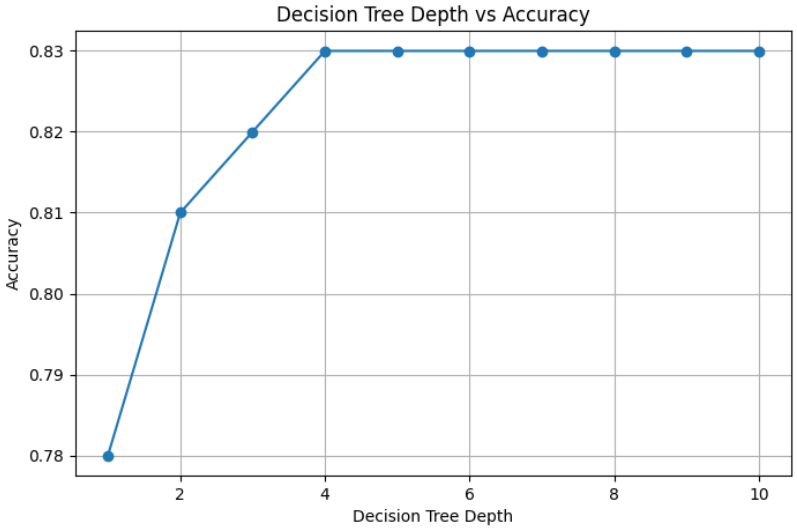


Figure 3: Random Forest Results and Confusion Matrix

Class	Prec.	Recall	F1	Supp.
F	0.00	0.00	0.00	7
N	0.90	0.99	0.94	801
Q	0.96	0.57	0.71	97
S	0.00	0.00	0.00	24
V	0.84	0.72	0.77	71
Acc.		0.90 (1000)		

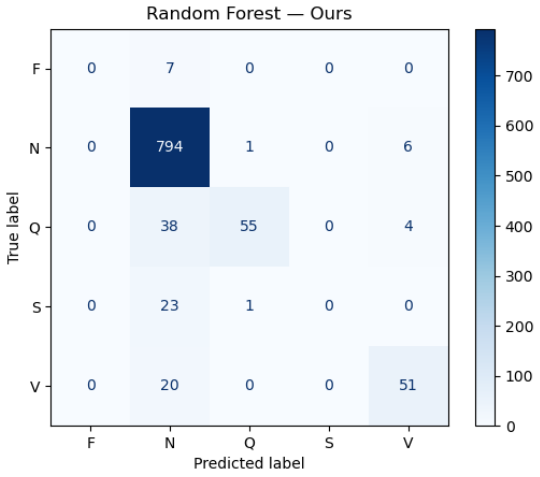
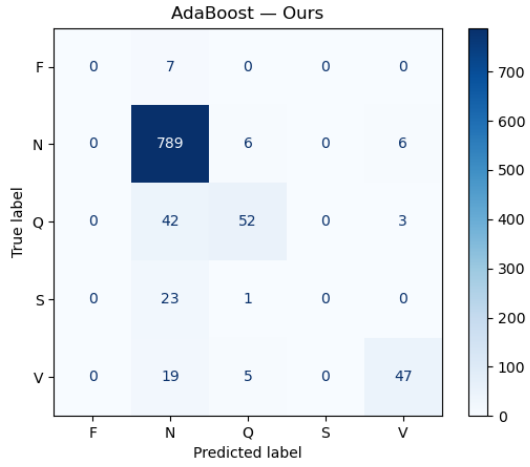


Figure 4: AdaBoost Results and Confusion Matrix

Class	Prec.	Recall	F1	Supp.
F	0.00	0.00	0.00	7
N	0.90	0.99	0.94	801
Q	0.81	0.54	0.65	97
S	0.00	0.00	0.00	24
V	0.84	0.66	0.74	71
Acc.		0.89 (1000)		



5.3 Discussion

Figure 2 shows diminishing returns for deeper single trees: accuracy rises quickly up to max depth 4, then flattens. This empirical curve justified fixing the tuned Decision Tree at **max_depth = 4, min_samples_split = 5**. Introducing controlled randomness in the Random Forest (60% bootstrap sampling and 60% feature subsampling) slightly improved robustness versus the full-sample/full-feature baseline, reducing overfitting signs while maintaining high Normal-class recall. Restricting depth to 4 across trees prevented individual over-complex trees from dominating. For AdaBoost, replacing depth-1 or depth-2 weak learners with depth-3 trees (min_samples_split = 2) yielded a modest accuracy gain without the instability observed when using depth-4 weak learners (which increased variance on scarce minority classes). This supports the principle that boosting benefits from moderately expressive, not overly deep, weak learners. Despite these tuning gains, severe class imbalance (especially S and F) remained the dominant performance bottleneck: minority-class recall and F1 stayed near zero because the training subset contained too few informative examples. Most misclassifications for minority beats defaulted to N, as reflected in confusion matrices. Thus, further hyperparameter refinement has limited impact until data imbalance is addressed. Overall, tuned ensemble configurations (Random Forest and AdaBoost) outperform the single tuned tree in aggregate metrics while still failing to generalize to the rarest classes, underscoring the need for targeted imbalance mitigation.

6 Conclusion and Future Work

In this project, we successfully implemented and evaluated three machine learning models for ECG beat classification using a well-defined set of engineered features. Our from-scratch implementations of Decision Tree, Random Forest, and AdaBoost classifiers performed comparably to their scikit-learn equivalents, demonstrating a solid understanding of the underlying algorithms. The ensemble methods proved most effective, with the Random Forest achieving the highest accuracy. Targeted hyperparameter tuning (tree depth, split thresholds, sampling and feature subsampling ratios, and weak learner depth) provided incremental improvements but could not overcome the scarcity of S and F class examples.

For future work, several improvements could be explored:

- **Address Class Imbalance:** Utilize advanced resampling techniques like SMOTE (Synthetic Minority Over-sampling Technique) to generate synthetic data for the minority classes (S and F), or use cost-sensitive learning.
- **Train on Full Dataset:** Leverage the entire dataset to provide more training examples, which is especially critical for the rare arrhythmia classes.
- **Explore Deep Learning:** Implement a CNN or RNN model to compare a feature-learning approach against our feature-engineering approach, which could potentially yield higher accuracy.

References

[1] Moody, G. B., & Mark, R. G. (2001). The impact of the MIT-BIH arrhythmia database. *IEEE Engineering in Medicine and Biology Magazine*, 20(3), 45-50.

[2] De Chazal, P., O'Dwyer, M., & Reilly, R. B. (2004). Automatic classification of heartbeats using ECG morphology and heartbeat interval features. *IEEE Transactions on biomedical engineering*, 51(7), 1196-1206.

[3] Acharya, U. R., Fujita, H., Oh, S. L., Hagiwara, Y., Tan, J. H., & Adam, M. (2017). Application of deep convolutional neural network for automated detection of myocardial infarction using ECG signals. *Information Sciences*, 415, 190-198.

[4] Kiranyaz, S., Ince, T., & Gabbouj, M. (2016). Real-time patient-specific ECG classification by 1-D convolutional neural networks. *IEEE Transactions on Biomedical Engineering*, 63(3), 664-675.

[5] Martis, R. J., Acharya, U. R., Mandana, K. M., Ray, A. K., & Chakraborty, C. (2013). ECG beat classification using PCA, LDA, and SVM. *Computers in biology and medicine*, 43(8), 1035-1044.

[6] Salha, A., Al-qudah, H., & Al-qadi, A. (2021). ECG arrhythmia classification using deep learning. *Sensors*, 21(11), 3823.